

■ 3.9.3 Numerical Integration

<code>N[Integrate[expr, {x, xmin, xmax}]]</code>	try to perform an integral exactly, then find numerical approximations to the parts that remain
<code>NIntegrate[expr, {x, xmin, xmax}]</code>	find a numerical approximation to an integral
<code>NIntegrate[expr, {x, xmin, xmax}, {y, ymin, ymax}, ...]</code>	multidimensional numerical integral $\int_{xmin}^{xmax} dx \int_{ymin}^{ymax} dy \dots expr$
<code>NIntegrate[expr, {x, xmin, x1, x2, ..., xmax}]</code>	do a numerical integral along a line, starting at <i>xmin</i> , going through the points <i>x_i</i> , and ending at <i>xmax</i>

Numerical integration functions.

This finds a numerical approximation to the integral $\int_0^\infty e^{-x^3} dx$.

```
In[1]:= NIntegrate[Exp[-x^3], {x, 0, Infinity}]
Out[1]= 0.89298
```

Here is the numerical value of the double integral $\int_{-1}^1 dx \int_{-1}^1 dy (x^2 + y^2)$.

```
In[2]:= NIntegrate[x^2 + y^2, {x, -1, 1}, {y, -1, 1}]
Out[2]= 2.66667
```

An important feature of `NIntegrate` is its ability to deal with functions that “blow up” at known points. `NIntegrate` automatically checks for such problems at the end points of the integration region.

The function $1/\sqrt{x}$ blows up at $x = 0$, but `NIntegrate` still succeeds in getting the correct value for the integral.

```
In[3]:= NIntegrate[1/Sqrt[x], {x, 0, 1}]
Out[3]= 2.
```

Mathematica can find the integral of $1/\sqrt{x}$ exactly.

```
In[4]:= Integrate[1/Sqrt[x], {x, 0, 1}]
Out[4]= 2
```

`NIntegrate` detects that the singularity in $\frac{1}{x}$ at $x = 0$ is not integrable.

```
In[5]:= NIntegrate[1/x, {x, 0, 1}]
NIntegrate::conv:
Numerical integral failed to reach specified
accuracy after 7 recursive subdivisions near
9.394176720306658 10-13 for variable number 1.
Out[5]= 111.92
```

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; <http://www.wolfram.com/>; 1-800-441-6284.

© 1988 Wolfram Research, Inc. Permission is hereby granted for web users to make one paper copy of this page for their personal use. Further reproduction, or any copying of machine-readable files (including this one) to any server computer, is strictly prohibited.

NIntegrate does not automatically look for singularities except at the end points of your integration region. When other singularities are present, **NIntegrate** may not give you the right answer for the integral. Nevertheless, in following its adaptive procedure, **NIntegrate** will often detect the presence of potentially singular behavior, and will warn you about it.

NIntegrate does not handle the singularity in $1/\sqrt{|x|}$ in the middle of the integration region. However, it warns you of a possible problem. In this case, the final result is numerically quite close to the correct answer.

```
In[6]:= NIntegrate[1/Sqrt[Abs[x]], {x, -1, 1}]
NIntegrate::conv:
  Numerical integral failed to reach specified
    accuracy after 7 recursive subdivisions near
    -0.015625 for variable number 1.
NIntegrate::conv:
  Numerical integral failed to reach specified
    accuracy after 7 recursive subdivisions near
    0.015625 for variable number 1.
Out[6]= 3.97439
```

If you know that your integrand has singularities at particular points, you can explicitly tell **NIntegrate** to deal with them.

NIntegrate[*expr*, {*x*, *xmin*, *x1*, *x2*, ..., *xmax*}] integrates *expr* from *xmin* to *xmax*, looking for possible singularities at each of the intermediate points x_i .

This again gives the integral $\int_{-1}^1 \frac{1}{\sqrt{|x|}} dx$, but now explicitly deals with the singularity at $x = 0$.

```
In[7]:= NIntegrate[1/Sqrt[Abs[x]], {x, -1, 0, 1}]
Out[7]= 4.
```

You can also use the list of intermediate points x_i in **NIntegrate** to specify an integration contour to follow in the complex plane. The contour is taken to consist of a sequence of line segments, starting at *xmin*, going through each of the x_i , and ending at *xmax*.

This integrates $\frac{1}{x}$ around the closed contour in the complex plane, going from -1, through the points $-i$, 1 and i , then back to -1.

```
In[8]:= NIntegrate[1/x, {x, -1, -I, 1, I, -1}]
Out[8]= 6.28319 I
```

The integral gives $2\pi i$, as expected from Cauchy's theorem.

```
In[9]:= N[ 2 Pi I ]
Out[9]= 6.28319 I
```

There are a number of ways you can control the operation of **NIntegrate**. First, you may want to specify the accuracy of the answers you are trying to get. If you use **Integrate**, and then apply **N** to get numerical results, then the second argument you give to **N** specifies the precision to use in the internal computations used to do the numerical integral. You can also specify this precision using the option

`WorkingPrecision -> n` for `NIntegrate`.

You should realize, however, that the option `WorkingPrecision` specifies only the precision to use for internal computations done by `NIntegrate`. The final answer that `NIntegrate` gives will almost always have a lower precision. You can nevertheless use `NIntegrate` to try and get an answer with a particular accuracy by setting the option `AccuracyGoal -> n`. The setting for `AccuracyGoal` gives the number of digits to the right of the decimal point that `NIntegrate` tries to get. In almost all cases, you should set `WorkingPrecision` to be at least as large as `AccuracyGoal`.

This evaluates $\int_1^2 \frac{1}{x} dx$ using 40 digit precision for internal computations.

```
In[10]:= NIntegrate[1/x, {x, 1, 2}, WorkingPrecision->40]
Out[10]= 0.69314718055994523673707961730813480957691
```

The result differs from the exact value of $\log(2)$ by about 1 part in 10^{20} .

```
In[11]:= % - N[Log[2], 40]
Out[11]= -7.268015250415004175849859 10-17
```

You can use the option `AccuracyGoal` to specify the number of digits of accuracy you want in the final answer. You often need to specify a slightly higher precision for internal computations in order to get a particular accuracy for the final result.

```
In[12]:= NIntegrate[1/x, {x, 1, 2}, WorkingPrecision->40,
AccuracyGoal->30]
Out[12]= 0.69314718055994530941723212145817656690973
```

Now the result is accurate to the full 30 digits you asked for.

```
In[13]:= % - N[Log[2], 30]
Out[13]= -2. 10-34
```

AccuracyGoal	6	number of digits of accuracy to try and get in the final answer
MinRecursion	1	minimum number of recursive subdivisions of the integration region
MaxRecursion	6	maximum number of recursive subdivisions
SingularityDepth	3	number of recursive subdivisions to use before doing a change of variables
Points	Automatic	number of sample points in each direction to use at first
WorkingPrecision	Precision[1.]	number of digits to use in internal computations

Options for NIntegrate.

When **NIntegrate** tries to evaluate a numerical integral, it samples the integrand at a sequence of points. If it finds that the integrand changes rapidly in a particular region, then it recursively takes more sample points in that region. The parameters **MinRecursion** and **MaxRecursion** specify the minimum and maximum number of levels of recursive subdivision to use. Increasing the value of **MinRecursion** guarantees that **NIntegrate** will use a larger number of sample points. **MaxRecursion** limits the number of sample points which **NIntegrate** will ever try to use. Increasing **MinRecursion** or **MaxRecursion** will make **NIntegrate** work more slowly. The option **Points** specifies how many sample points **NIntegrate** should use before it starts recursive subdivision. **SingularityDepth** specifies how many levels of recursive subdivision **NIntegrate** should try before it concludes that the integrand is “blowing up”, and does a change of variables.

With the default settings for all options, **NIntegrate** misses the peak in $\exp(-x^2)$ near $x = 0$, and gives the wrong answer for the integral.

```
In[14]:= NIntegrate[Exp[-x^2], {x, -500, 500}]
Out[14]= 7.97921 10-42
```

With the option `MinRecursion->2`, `NIntegrate` samples enough points that it notices the peak around $x = 0$. With the default setting of `MaxRecursion`, however, `NIntegrate` cannot use enough sample points to guarantee an accurate answer.

```
In[15]:= NIntegrate[Exp[-x^2], {x, -500, 500},
             MinRecursion->2]
NIntegrate::conv:
  Numerical integral failed to reach specified
    accuracy after 7 recursive subdivisions near
    -7.8125 for variable number 1.
NIntegrate::conv:
  Numerical integral failed to reach specified
    accuracy after 7 recursive subdivisions near
    7.8125 for variable number 1.
Out[15]= 1.77563
```

With this setting of `MaxRecursion`, `NIntegrate` can get an accurate answer for the integral.

```
In[16]:= NIntegrate[Exp[-x^2], {x, -500, 500},
             MinRecursion->2, MaxRecursion->10]
Out[16]= 1.77245
```

Another way to solve the problem is to make `NIntegrate` break the integration region into several pieces, with a small piece that explicitly covers the neighborhood of the peak.

```
In[17]:= NIntegrate[Exp[-x^2], {x, -500, -10, 10, 500}]
Out[17]= 1.77245
```